

Invited: The Magnificent Seven Challenges and Opportunities in Domain-Specific Accelerator Design for Autonomous Systems

Sabrina M. Neuman*
Boston University
sneuman@bu.edu

Brian Plancher*
Barnard College, Columbia University
bplancher@barnard.edu

Vijay Janapa Reddi*
Harvard University
vj@eecs.harvard.edu

Abstract

The end of Moore's Law and Dennard Scaling has combined with advances in agile hardware design to foster a golden age of domain-specific acceleration. However, this new frontier of computing opportunities is not without pitfalls. As computer architects approach unfamiliar domains, we have seen common themes emerge in the challenges that can hinder progress in the development of useful acceleration. In this work, we present the Magnificent Seven Challenges in domain-specific accelerator design that can guide adventurous architects to contribute meaningfully to novel application domains. Although these challenges appear across domains ranging from ML to genomics, we examine them through the lens of autonomous systems as a motivating example in this work. To that end, we identify opportunities for the path forward in a successful domain-specific accelerator design from these challenges.

CCS Concepts

- **Hardware** → **Hardware accelerators; Best practices for EDA;**
- **Computer systems organization** → **Robotics.**

Keywords

Domain-Specific Architectures, Accelerators, Autonomous Systems

ACM Reference Format:

Sabrina M. Neuman, Brian Plancher, and Vijay Janapa Reddi. 2024. Invited: The Magnificent Seven Challenges and Opportunities in Domain-Specific Accelerator Design for Autonomous Systems. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3649329.3663515>

1 Introduction

In the first quarter of this century, computing hardware designers were faced with both the limitations of technology scaling for performance [1], and the ensuing conflagration of on-chip power density [2]. Fortunately, the challenges of the Dark Silicon era [3] have transitioned into a prosperous period of specialized accelerator design [4, 5]. Emerging enabling technology and tools for agile and democratized hardware design are powering an exciting *land*

*All authors contributed equally to this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

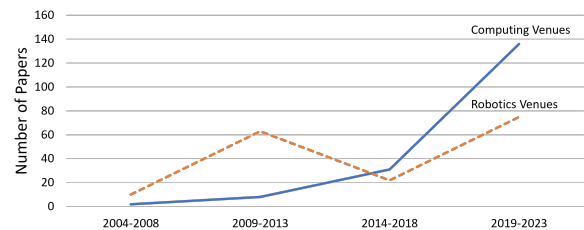
DAC '24, June 23–27, 2024, San Francisco, CA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3663515>

Growth of the Topic of Accelerators for Autonomous Systems



Computing Keywords: Accelerator & Robot | Autonomous System | UAV | Drone | Autonomous Vehicle | Self-Driving, in DAC, ISCA, MICRO, HPCA, and ASPLOS. Robotics Keywords: ASIC | FPGA, in ICRA, IROS, RSS, and RA-L.

Figure 1: Mention of accelerators for autonomous systems in top-tier computing and robotics venues, from Google Scholar.

rush in applying computer architecture to application domains that are not well-served by existing solutions.

Examples of promising domains for acceleration have ranged from the popular machine learning (ML) space [6–13] to emerging work in robotics [14–23] (Fig. 1) and genomics [24–26]. While these solutions have provided substantial speedups and improvements in their target applications and kernels of interest, as of yet, many of these artifacts have only had limited impact on real world production systems. In ML, apart from the success of the Google TPU [10], large data centers still run the latest NVIDIA GPUs for ML training and inference. Similarly, while early work in acceleration for robotics and genomics have delivered promising results, most commercial solutions exclusively leverage CPUs and GPUs.

In this work, we present a collection of key challenges in domain-specific accelerator design, viewed through the lens of autonomous systems, with complementary examples drawn from the more mature ML acceleration literature. Reflecting on our experience working in this space, we find that there are common themes in the pitfalls that drive the lack of technology transfer from successful research prototypes to deployed real-world solutions. We identify examples of current work that overcomes these challenges, providing examples of successful strategies for future work.

2 The Magnificent Seven Challenges

In the following, we examine each of the seven challenges in detail. We outline the pitfalls of the conventional approaches to accelerator design, and survey instructive success stories in recent work.

2.1 Build Bridges: Engage with Domain Experts

Pitfall: Interact with domains exclusively through benchmarks published in computer systems, without input from domain experts.

A common pitfall in the development of domain-specific architectures (DSAs) is the lack of direct engagement with domain experts, leading to a misunderstanding of the user's needs. This often results in the development of solutions that, while technically impressive,

fail to address practical user requirements. This mis-match can manifest *across all stages of design and development*, from initially targeting the wrong bottlenecks to delivering an unusable artifact.

A DSA might be optimized for a specific algorithm that, unbeknownst to the developers, has little impact in the real world. For example, simultaneous localization and mapping (SLAM) algorithms for autonomous systems are rapidly evolving— a 2023 survey identified 24 representative “active SLAM” algorithmic approaches [27]. Without ongoing feedback from domain experts, an obsolete approach might be accidentally accelerated. In the past decade, there have been teams that have avoided this pitfall through the successful *engagement of computing and robotics experts* to produce interdisciplinary work together, e.g., the Navion accelerator for visual-inertial odometry [16, 28], and the motion planning accelerators [14, 29] that led to the startup Realtime Robotics [30]. Collaborations with industry are another way to anchor design work in real-world expertise, e.g., NVIDIA’s CuRobo library [31].

After design, the effective deployment of DSAs requires interfaces and wrappers optimized for existing users and workflows. For autonomous systems, a popular solution has been to integrate into existing open-source frameworks, e.g., the Robot Operating System (ROS) middleware [32]. For example, a large body of work in accelerating motion planning in software (e.g., [33]) has built upon the ROS-integrated Open Motion Planning Library (OMPL) [34].

Finally, note that designers must not only address current domain needs, but anticipate potentially *unanticipated* future needs (Sec. 4).

2.2 Measure Twice, Cut Once: Metrics Matter

Pitfall: Only focus on improving throughput or energy-delay product.

The choice of metrics for evaluating DSA performance can significantly influence the direction and outcomes of a project. For example, ML practitioners might prioritize accuracy, whereas accelerator designers might focus on optimizing throughput. This discrepancy can lead to solutions that, while improving throughput, degrade model accuracy, rendering them less useful or necessitating longer training times to achieve comparable accuracy. Beyond accuracy, pursuing traditional high performance computing metrics, e.g., *tera-operations per second per watt* (TOPS/W), in isolation from *system-level metrics* (e.g., off-chip bandwidth) has been demonstrated to be misleading in ML accelerators [35]. Autonomous systems have analogous complex tradeoffs, for example, depending on control algorithm and hyperparameter choices [36]. Metrics measuring, e.g., time-to-accuracy and holistic system performance, rather than raw throughput or energy efficiency, are needed.

2.3 “Widgetism”: Avoid Over-Specialization

Pitfall: A cycle of pick one slow algorithm, lower it to an ASIC, repeat.

DSAs have the potential to revolutionize computing by providing highly optimized solutions for specific domains. However, there is a significant risk of falling into the trap of “widgetism,” where the focus becomes too narrow and specialized, limiting the broader impact and adaptability of designs. The key to overcoming this is to identify and support *cross-cutting kernels* that represent fundamental operations that are applicable across tasks (e.g., motion planning in autonomous systems [34]). In ML, for example, hardware acceleration of common kernels, such as sparse tensor algebra [37], can

support a wide variety of different user tasks with the same accelerator architectures. By focusing on such kernels, DSAs can provide significant performance improvements while maintaining flexibility. Over-specialization also occurs when DSAs are only *evaluated* on a narrow set of tasks. This incentivizes high-performance “widgets” that may be overfit to unrealistic or uncommon use cases.

2.4 Pump the Brakes: Do Not Always Accelerate

Pitfall: Assume accelerators always improve total system performance.

The drive to accelerate every aspect of computation without considering the broader system context can lead to inefficiencies, particularly in complex environments. Accelerators, while powerful, are not free: they consume shared resources and can introduce complexities in system scheduling and resource allocation [35]. It is crucial to evaluate the impact of accelerators within the context of the entire system. For example, recent work on choosing onboard processors for UAVs [38] found that for overall mission performance, it was important to balance the processing rates of compute with sensor input rates— and, in fact, *over-provisioning compute could have disastrous effects* on the weight and battery life of the total system. For complex autonomous systems, in some cases, the best decision might be to throttle, or even forego acceleration in favor of maintaining system harmony and efficiency.

2.5 Chips and Salsa: Acceleration Beyond ASICs

Pitfall: Focus on ASICs, leaving software, GPUs, and FPGAs behind.

Software solutions on CPUs and GPUs, and firmware on FPGAs are all in deployment in real commercial systems—we must not overlook opportunities to accelerate autonomous systems with these platforms. Programmable devices like GPUs and FPGAs offer flexibility and have benefited a wide range of applications. Notably, utilization of GPUs has enabled rapid growth in AI and ML in the past decade. In autonomous systems, recent work on software-only optimizations that leveraged vectorization on the CPU achieved up to 500x speedups over state-of-the-art for certain motion planning problems [33]. Work on software libraries for autonomous systems algorithms (e.g., [39]) can extend similar benefits to GPUs. Even as new domain-specific ASICs are introduced into existing systems, they will likely be part of a complete heterogeneous computing solution, balancing the performance of dedicated hardware and the flexibility afforded by software and programmable platforms.

2.6 Forest vs. Trees: Take an End-to-End View

Pitfall: A narrow scope: acceleration begins and ends with compute.

Focusing exclusively on optimizing individual kernels (even cross-cutting ones) without considering the end-to-end system *beyond the computing system* (e.g., including data, I/O, sensors, actuators, and real-world effects like reliability and robustness to noise) can lead to DSAs that improve theoretical performance but fail to deliver real-world benefits. This effect has been seen in the “AI Tax” of datacenter-scale ML systems, where costs of user data marshalling and network I/O transfer are substantial compared to edge device inference. Autonomous systems are also complex to deploy, and considerations such as weight, battery technology, sensor and actuator

quality, are layered on top of traditional computing design concerns [38, 40, 41]. An example of a promising approach to tackling this full deployment complexity is to build simulation and testing environments that model the full system and its interactions with the environment, e.g., the MAVBench and RoSE simulators for UAV acceleration [42, 43], and ILLIXR for AR/VR platforms [44].

2.7 Design Global: Sustainability and Impact

Pitfall: Design compute in isolation from its global and societal impact.

Accelerator design has often viewed computing devices as isolated entities, focusing on performance metrics within the confines of the hardware itself. However, we must consider the environmental and societal impact of computing design at scale. For example, recent work makes a compelling case that autonomous vehicles should be viewed as “datacenters on wheels” [45] and predicts that future global-scale self-driving systems may require computational resources that exceed traditional datacenters. Similarly, recent ML research noted that the higher efficiency of cloud-based training compared to on-device processing implies that choosing to train ML models on edge devices can lead to a greater increase in carbon emissions [46]. These results indicate that the design of accelerators deployed to the edge must take into consideration global factors in their manufacturing, deployment scale, and operation.

3 Future Directions & Opportunities

To address these key challenges, domain-specific accelerator design requires close collaboration between domain experts and hardware designers, adaptability to evolving algorithmic landscapes, and a holistic view of the entire system, including global impact at scale.

3.1 Enabling Technologies and Methodologies

Agile Design Tools. One key opportunity lies in the development of agile design tools that streamline the creation and customization of DSAs. There has been a growing body of work on hardware compiler frameworks and high-level synthesis (HLS) [47–50], and these tools should continue to be leveraged and augmented to power domain-specific design. To be effective in the autonomous systems space, these tools should operate at a high level of abstraction, allowing users to specify accelerator functionality through intuitive methods such as domain-specific languages (DSLs) or graphical interfaces, enabling domain experts to directly participate in the design process. Moreover, agile design tools should strive to bridge the gap between high-level algorithmic specifications and low-level hardware implementations, e.g., with formal verification techniques to prove correctness of generated hardware from a specification—which is crucial in the safety-critical field of autonomous systems.

End-to-End Modeling and Simulation. To design effective DSAs, it is essential to consider the entire system in which they will be deployed, rather than focusing solely on the accelerator in isolation. This requires end-to-end modeling and simulation tools that can capture the complex interactions between the accelerator, the rest of the computing system, and the physical environment. A key aspect of end-to-end modeling is the ability to accurately simulate the data flow through the system, from sensors and input devices to the accelerator and back to actuators or output devices (e.g., the MAVBench [42] and RoSE [43] simulators). This simulation

should take into account the bandwidth and latency constraints, processing delays, and power and thermal characteristics of the system. Finally, end-to-end modeling should consider the physical environment in which the system will be deployed. For example, for an autonomous vehicle, the model should take into account factors such as vehicle dynamics, sensor noise, and weather conditions.

Machine Learning for System Design. ML has the potential to support the design and optimization of accelerators for autonomous systems and other domains. By leveraging the ability of ML models to learn complex patterns and make data-driven decisions, it is possible to automate many tedious and time-consuming aspects of the accelerator design process. One promising use of ML is design space exploration. Given an algorithm specification and a set of hardware constraints, an ML model can be trained to search the space of possible hardware configurations and identify the most promising candidates considering the full-system [38], saving time and discovering unconventional designs that experts might overlook.

3.2 Fostering a Robust Research Ecosystem

Cross-Domain Collaboration. Accelerator design is inherently multidisciplinary. To facilitate collaboration between diverse communities, it is essential to establish dedicated forums for knowledge sharing and exchange including specialized interdisciplinary workshops, conferences, research centers, and consortia. Domain experts can also be more easily integrated into the design process through the inclusive design of tools and interfaces such as DSLs (Sec. 3.1).

Open-Source Resources. A key element of a robust research ecosystem is the availability of open-source resources, including software tools, hardware designs, and datasets. Such resources lower the barriers to entry for new researchers, enable reproducibility and comparability of results, accelerating innovation. Open-source datasets and benchmarks that are representative of real-world workloads enable researchers to evaluate their designs under realistic conditions and compare their results against those of other researchers. Examples such as the ILLIXR AR/VR workload suite [44] and the RoSE framework for RTL and physics simulation [43] demonstrate the open-sourcing of software to fairly evaluate the performance and power efficiency of accelerator designs.

Standardized Benchmarks and Metrics. To enable meaningful comparison and evaluation of accelerator designs, it is essential to establish widely-accepted, standardized benchmarks and metrics. These benchmarks and metrics should not only evaluate domain performance, but also energy efficiency, cost, and other key characteristics of accelerator designs. One example of such a benchmark suite for ML accelerators is MLPerf [51], which provides a set of reference workloads and evaluation criteria for measuring the performance of ML training and inference systems. Finally, standardized benchmarks and metrics can also play a key role in driving innovation and guiding research priorities by identifying the most important and challenging workloads and use cases.

3.3 Sustainable & Responsible Hardware Design

Strategic Deployment of Accelerators. While accelerators can provide significant benefits for specific workloads, it is important to consider their deployment scenarios (Sec. 2.7). This includes factors such as the cost of development and deployment (e.g., using

emissions modeling tools [52]), the scalability of the solution, and the potential for reuse across applications and domains. This helps avoid the proliferation of short-lifespan, over-specialized hardware, which contributes to electronic waste. For example, chiplet technology can help by providing a modular design approach that promotes the reuse and repurposing of hardware components [53].

Lifecycle Analysis & End-of-Life Management. Lifecycle analysis (LCA) is useful for assessing the environmental impacts of a product or system throughout its entire lifespan, from raw materials extraction to end-of-life management. By conducting LCAs for accelerator designs, we can identify opportunities for reducing the environmental footprint of hardware, such as selecting low-impact materials, optimizing manufacturing processes, or designing for easier disassembly and recycling. Effective end-of-life management is also key to minimize the environmental impact of accelerators, from recycling, to recovering valuable materials, to the safe disposal of hazardous materials. Expanding the scope of LCA to encompass entire autonomous systems (e.g., self-driving vehicle fleets [45]) would be a step towards sustainable automation in the future.

4 Conclusion

By understanding the common challenges of domain-specific design for autonomous systems and beyond, we can deploy accelerators that provide a positive broader impact both on their target applications as well as the larger world. Finally, it is important to remember that Amdahl's Law is a moving target. Recalling Henry Ford's possibly apocryphal [54] remark on innovation¹, accelerator designers should recognize that *anticipating the future needs* of a domain requires a constant re-examination of the fundamental benchmarks and assumptions, and dynamic analysis to continually identify new opportunities over time. Incorporating feedback mechanisms into the design process ensures that useful contributions continue to be made as societal needs and technological landscapes evolve.

References

- [1] K. Asanovic, *et al.*, "The landscape of parallel computing research: A view from Berkeley," *University of California Technical Report*, 2006.
- [2] H. Esmaeilzadeh, *et al.*, "Dark silicon and the end of multicore scaling," in *ISCA*, 2011.
- [3] M. B. Taylor, "Is dark silicon useful? harnessing the four horsemen of the coming dark silicon apocalypse," in *DAC*, 2012.
- [4] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *CACM*, 2019.
- [5] W. J. Dally, *et al.*, "Domain-specific hardware accelerators," *CACM*, 2020.
- [6] S. K. Esser, *et al.*, "Backpropagation for energy-efficient neuromorphic computing," *NeurIPS*, 2015.
- [7] Y.-H. Chen, *et al.*, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *JSSC*, 2016.
- [8] Y. Chen, *et al.*, "Diannao family: energy-efficient hardware accelerators for machine learning," *CACM*, 2016.
- [9] A. Parashar, *et al.*, "Scnn: An accelerator for compressed-sparse convolutional neural networks," *SIGARCH News*, 2017.
- [10] N. P. Jouppi, *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *ISCA*, 2017.
- [11] A. Ankit, *et al.*, "Puma: A programmable ultra-efficient memristor-based accelerator for machine learning inference," in *ASPLOS*, 2019.
- [12] A. Gondimalla, *et al.*, "Sparten: A sparse tensor accelerator for convolutional neural networks," in *MICRO*, 2019.
- [13] E. Qin, *et al.*, "Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in *HPCA*, 2020.
- [14] S. Murray, *et al.*, "The microarchitecture of a real-time robot motion planning accelerator," in *MICRO*, 2016.
- [15] J. Sacks, *et al.*, "Robox: an end-to-end solution to accelerate autonomous control in robotics," in *ISCA*, 2018.
- [16] A. Suleiman, *et al.*, "Navion: A 2-mw fully integrated real-time visual-inertial odometry accelerator for autonomous navigation of nano drones," *JSSC*, 2019.
- [17] B. Asgari, *et al.*, "Pisces: power-aware implementation of slam by customizing efficient sparse algebra," in *DAC*, 2020.
- [18] Y. Liu, *et al.*, "Hardware acceleration of monte-carlo sampling for energy efficient robust robot manipulation," in *FPL*, 2020.
- [19] Y. Han, *et al.*, "Dadu series-fast and efficient robot accelerators," in *ICCAD*, 2020.
- [20] W. Liu, *et al.*, "Archytas: A framework for synthesizing and dynamically optimizing accelerators for robotic localization," in *MICRO*, 2021.
- [21] S. M. Neuman, *et al.*, "Robomorphic computing: a design methodology for domain-specific accelerators parameterized by robot morphology," in *ASPLOS*, 2021.
- [22] M. Bakhshalipour, *et al.*, "Racod: Algorithm/hardware co-design for mobile robot path planning," in *ISCA*, 2022.
- [23] S. M. Neuman, *et al.*, "Roboshape: Using topology patterns to scalably and flexibly deploy accelerators across robots," in *ISCA*, 2023.
- [24] Y. Turakhia, *et al.*, "Darwin: A genomics co-processor provides up to 15,000 x acceleration on long read assembly," *ACM SIGPLAN Notices*, 2018.
- [25] M. Doblas, *et al.*, "Gmx: Instruction set extensions for fast, scalable, and efficient genome sequence alignment," in *MICRO*, 2023.
- [26] D. S. Cali, *et al.*, "Segram: A universal hardware accelerator for genomic sequence-to-graph and sequence-to-sequence mapping," in *ISCA*, 2022.
- [27] J. A. Placed, *et al.*, "A survey on active simultaneous localization and mapping: State of the art and new frontiers," *TRO*, 2023.
- [28] Z. Zhang, *et al.*, "Visual-inertial odometry on chip: An algorithm-and-hardware co-design approach," in *RSS*, 2017.
- [29] S. Murray, *et al.*, "Robot motion planning on a chip," in *RSS*, 2016.
- [30] "Realtime Robotics." [Online]. Available: <https://rtr.ai/>
- [31] B. Sundaralingam, *et al.*, "Curobo: Parallelized collision-free robot motion generation," in *ICRA*, 2023.
- [32] M. Quigley, *et al.*, "Ros: an open-source robot operating system," in *ICRA OSS Workshop*, 2009.
- [33] W. Thomason, *et al.*, "Motions in microseconds via vectorized sampling-based planning," *arXiv*, 2023.
- [34] I. A. Sucan, *et al.*, "The open motion planning library," *RAM*, 2012.
- [35] V. Sze, *et al.*, "How to evaluate deep neural network processors: Tops/w (alone) considered harmful," *SSC-M*, 2020.
- [36] M. Moll, *et al.*, "Hyperplan: A framework for motion planning algorithm selection and parameter optimization," in *IROS*, 2021.
- [37] S. Dave, *et al.*, "Hardware acceleration of sparse and irregular tensor computations of ml models: A survey and insights," *Proceedings of the IEEE*, 2021.
- [38] S. Krishnan, *et al.*, "Automatic domain-specific soc design for autonomous unmanned aerial vehicles," in *MICRO*, 2022.
- [39] B. Plancher, *et al.*, "Grid: Gpu-accelerated rigid body dynamics with analytical gradients," in *ICRA*, 2022.
- [40] R. Hadidi, *et al.*, "Quantifying the design-space tradeoffs in autonomous drones," in *ASPLOS*, 2021.
- [41] S. M. Neuman, *et al.*, "Tiny robot learning: Challenges and directions for machine learning in resource-constrained robots," in *AICAS*, 2022.
- [42] B. Boroujerdian, *et al.*, "Mavbench: Micro aerial vehicle benchmarking," in *MICRO*, 2018.
- [43] D. Nikiforov, *et al.*, "Rosé: A hardware-software co-simulation infrastructure enabling pre-silicon full-stack robotics soc evaluation," in *ISCA*, 2023.
- [44] M. Huzaifa, *et al.*, "Illixr: Enabling end-to-end extended reality research," in *IISWC*, 2021.
- [45] S. Sudhakar, *et al.*, "Data centers on wheels: emissions from computing onboard autonomous vehicles," *IEEE Micro*, 2022.
- [46] D. Patterson, *et al.*, "Energy and emissions of machine learning on smartphones vs. the cloud," *CACM*, 2024.
- [47] S. Eldridge, *et al.*, "Mir as hardware compiler infrastructure," in *Workshop on Open-Source EDA Technology (WOSET)*, 2021.
- [48] R. Nigam, *et al.*, "A compiler infrastructure for accelerator generators," in *ASPLOS*, 2021.
- [49] F. Winterstein, *et al.*, "High-level synthesis of dynamic data structures: A case study using vivado hls," in *FPT*, 2013.
- [50] R. Nikhil, "Bluespec system verilog: efficient, correct rtl from high level specifications," in *MEMOCODE*, 2004.
- [51] V. J. Reddi, *et al.*, "Mlperf inference benchmark," in *ISCA*, 2020, pp. 446–459.
- [52] U. Gupta, *et al.*, "Architectural co2 footprint tool: Designing sustainable computer systems with an architectural carbon modeling tool," *IEEE Micro*, 2023.
- [53] C. C. Sudarshan, *et al.*, "Greenfpga: Evaluating fpgas as environmentally sustainable computing solutions," *HPCA*, 2024.
- [54] P. Vlaskovits, "Henry ford, innovation, and that 'faster horse' quote," *Harvard Business Review*, 2011.

¹The automobile tycoon is often credited with the quote, "If I had asked people what they wanted, they would have said faster horses."